Weekly meeting (260127)

Overview

OURS: Adaptive Query Rewriting

- Adaptive: 기존 retrieval에서 못가져왔던 애들을 corpus tree를 통해서 가져온다는 점에서 adaptive
- vs REPAIR: Adaptive reranking
  - Query rewriting (QR)
    - REPAIR: Query plan. Planning answer to the question
    - Ours: Abstraction. Principles & evidence that support the answer
  - Corpus graph
    - REPAIR: parallel graph
      - documents with similar meaning are connected
    - Ours: hierarchical tree graph
      - While LLM lacks domain-specific knowledge, if we can match query with the branch node, gold document will be under the branch node.

- How to leverage corpus tree for better query rewriting?
  - Ineffective: Branch nodes as a context for a rewriter
  - Effective: As an adaptive retrieval

Corpus graph → Query rewriting

Method

- Which contexts to show for query rewrite
  - (1) Iterative query rewriting with corpus feedback
    - Q rewrite w/o documents → traverse depth=1 nodes → Q rewrite with traversed nodes → traverse depth=2 nodes → Q rewrite with traversed nodes → traverse depth=3 …
  - (1-1) feedback = branch
  - (1-2) feedback = leaf

	biology	psychology	Avg
(1-1)	59.80	**43.70**	51.8
(1-2)	**63.02**	42.50	**52.8**

Corpus graph → Query rewriting

Method

- Applying query rewrite with branch nodes for initial query rewriting
  - Assumption: Since branch nodes are abstracted documents, they will guide initial direction of query abstraction
  - (2-1) query rewriting **w/ branch nodes** → retrieval → traverse
  - (2-2) query rewriting **w/ all nodes** → retrieval → traverse

	biology	**psychology**	**Avg**
(2-1)	**61.73**	43.90	52.82
(2-2)	60.45	**45.73**	**53.09**

Corpus graph → Query rewriting

Experiment: Branch nodes → automatically build query abstraction category

- Originally, fixed abstraction categories (theory, entity, example, others)
- LLM extracts abstraction categories from branch nodes, then fill contents from leaf nodes

- Result: leaf node 보여주는 게 더 낫다

Methods	biology	psychology
schema w/ all nodes	58.75	45.93
schema w/ branch nodes	57.95	44.06
schema w/ leaf nodes	60.57	45.55

"

```
"```json\n"
"{\n"
"  \"Reasoning\": \"e.g. Theory 'Poisson' was found in Doc 1. However, Entity 'Submarine' was NOT found; docs discuss 'Bees'.\",\n"
"  \"Actions\": {\n"
"    \"Theory\": \"EXPLOIT\",\n"
"    \"Entity\": \"EXPLORE\",\n"
"    \"Example\": \"EXPLORE\"\n"
"    \"Other\": \"EXPLOIT\"\n"
"  }\n"
"}\n"
"```\n\n"
```

Corpus graph → Query rewriting w/ adaptive retrieval

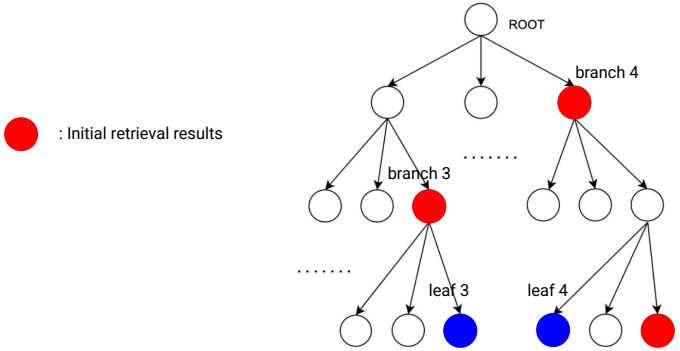
Adaptive: 기존 retrieval에서 못가져왔던 애들을 corpus tree를 통해서 가져온다는 점에서 adaptive

- (Last week FB) LLM이 직접 답을 생성하는 건 못할 수 있음. 세부적인 지식들을 모델이 다 알지는 못함. 하지만 지식이 abstraction되어 있다면 이에 대해 정답 판단하는 능력은 모델이 갖고 있다고 전제할 수 있음.

- Hypothesis: In the absence of domain-specific knowledge, if an abstractive concept matches at the branch level, the gold document likely exists under that branch

Method

- If the Top-5 results are
  - [leaf 1, leaf 2, branch 3 , branch 4 , leaf 5], convert them to
  - [leaf 1, leaf 2, best-leaf-under-branch 3, best-leaf-under-branch 4, leaf 5]
- Best-leaf is retrieved greedily.
  - Best of depth-1 children → best of depth-2 children → …



Corpus graph → Query rewriting w/ adaptive retrieval

- (1) ≅ (2): Naively merging query rewriting with corpus tree is ineffective
- (3) < (4): query abstraction is necessary
- (4) < (5): Corpus tree boosts performance
  - On average: True. For subcategories, False

	Tree	Method	bio	earth	**econ**	**psych**	robot	avg
Baselines	O	(1) Tree-traversal (LATTICE)	40.89	41.09	28.06	36.33	27.1	34.69
	O	(2) ThinkQE + tree-traversal	38.26	45.88	26.01	33.99	27.87	34.4
	X	(3) ThinkQE + Iterative rewriting	49.82	52.63	27.01	40.8	23.93	38.84
Ours	X	(4) AbstractQE + Iterative rewriting	60.23	58.57	**28.57**	**44.54**	26.89	43.76
	O	(5) AbstractQE + adaptive retrieval	**63.41**	**59.1**	26.56	43.65	**27.88**	**44.12**

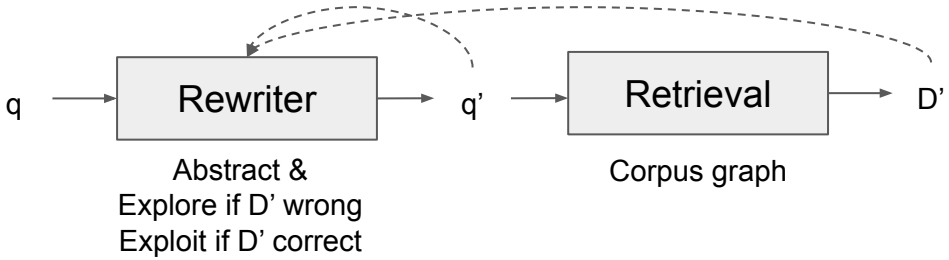
Model: Qwen3-4B-2507-Instruct

Corpus graph → Query rewriting w/ adaptive retrieval

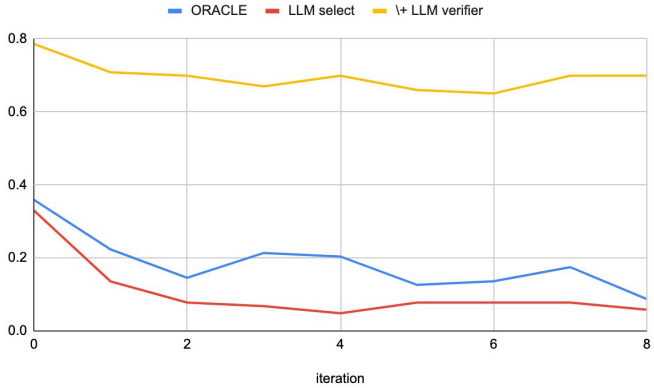
- (1) ≅ (2): Naively merging query rewriting with corpus tree is ineffective
- (3) < (4): query abstraction is necessary
- (4) < (5): Corpus tree boosts performance
  - On average: True. For subcategories, False
- (6): Oracle adaptive retrieval: best-leaf-under-branchX 가 항상 gold doc을 가져오는 경우
- (7): Oracle Explore / Exploit selection
  - 두 query rewrite에서 항상 더 좋은 rewrite을 선택하는 경우

	Tree	Method	bio	earth	econ	psych	robot	avg
Baselines	O	(1) Tree-traversal	40.89	41.09	28.06	36.33	27.1	34.69
	O	(2) = (1)+(3) ThinkQE + tree-traversal	38.26	45.88	26.01	33.99	27.87	34.4
	X	(3) ThinkQE + Iterative rewriting	49.82	52.63	27.01	40.8	23.93	38.84
Ours	X	(4) AbstractQE + Iterative rewriting	60.23	58.57	28.57	44.54	26.89	43.76
	O	(5) AbstractQE + adaptive retrieval	63.41	59.1	26.56	43.65	27.88	44.12
Oracle		(6) Oracle adaptive retrieval	61.09	58.88	31.18	46.95	31.83	45.99
		(7) Oracle Exploit / Explore	**68.82**	**65.65**	**36.09**	**50.54**	**32.46**	**50.71**

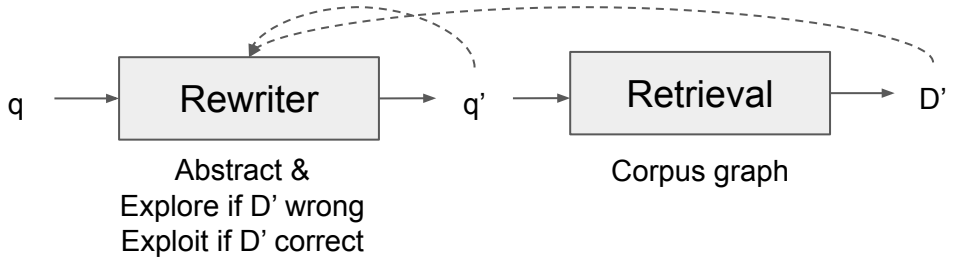
Direction



- Explore/Exploit: Solely on LLM decision is unstable
  - Performance gap: 44.12 < 50.71
  - LLM verifier fails: Given (D’_explore, q’_explore, D’_exploit, q’_exploit), LLM selects which one to pass to next iteration
    - Distribution gap becomes severe w/ oracle selection



TODO



External signal to help exploit / explore

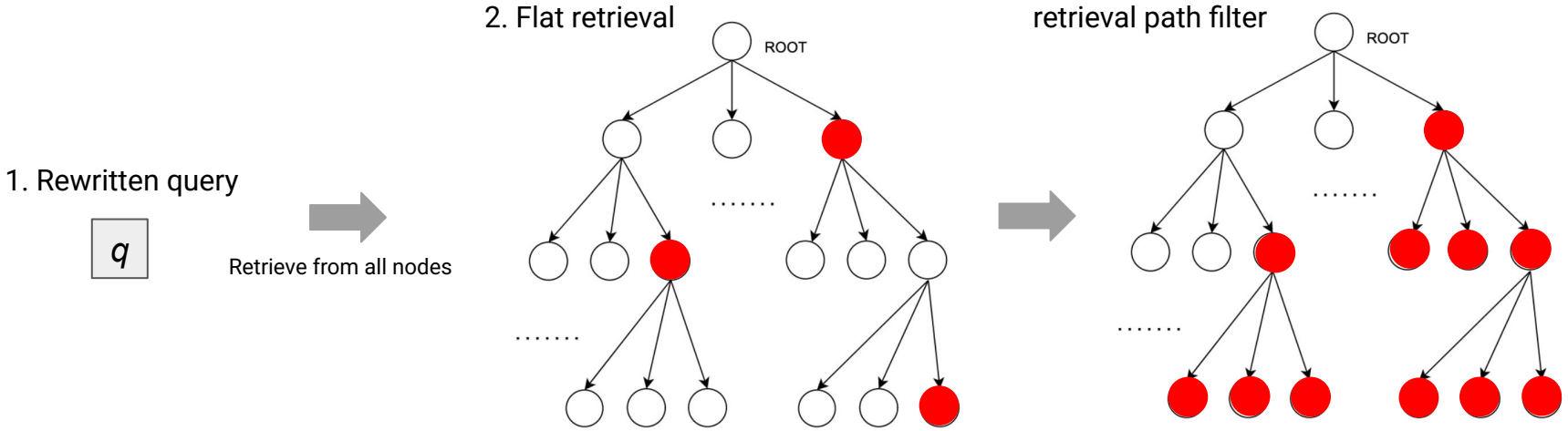
- Comparing retrieval score for each query and choose one
- Explore, exploit 둘 다 생성한 후 겹치는 문서는 유지, 나머지 문서에 대해 rerank [1,2]
- Corpus graph
  - Corpus 환경 전체 정보를 주기 위해 필요
  - e.g. When explore, 현재 leaf들이 속한 branch와 겹치지 않는 branch 후보들을 함께 제공
  - e.g. When exploit, 탐색 공간 제한

[1] Li et al. (2024), DMQR-RAG: Diverse Multi-Query Rewriting for Retrieval-Augmented Generation
[2] Kostric & Balog (2024), A Surprisingly Simple yet Effective Multi-Query Rewriting Method for Conversational Passage Retrieval

Appendix

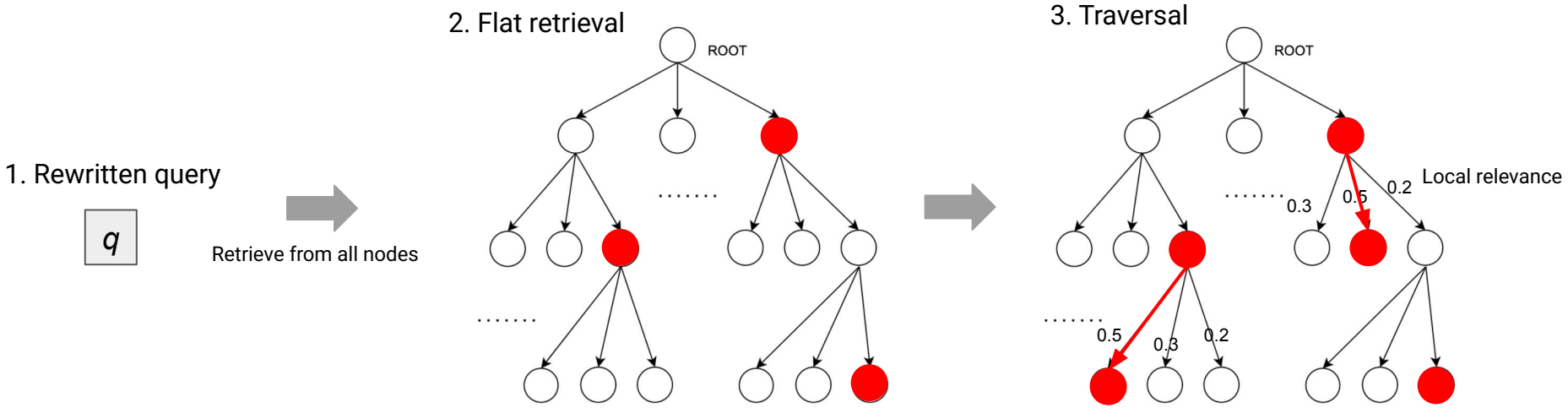
[Recap] Leveraging corpus abstraction tree

- 정확한 구현: flat retrieval & branch nodes as traversal constraints
  - Iteration 1: Scan depth 2 → Iteration 2: scan depth 3 → Iteration 3: scan depth 4 → Iteration 4: scan depth 5 (=root node)
  - RRF with leaf-only retrieval
  - Iteration 4 = peak and performance drops



[Recap] Leveraging corpus abstraction tree

-
- 실제로 아래 구현처럼 구현하면 성능 하락 (기존 score계산에서 문제 생기는듯)
-



Agentic search

- Agentic search
  - State: 각 level별 확보된 정보
  - Action: 비어있는 level에 대해 query rewrite (exploration) & 정보량 많은 level에 대해서 query refine (exploitation)
- Prompts

1. Router

```
"Inputs:\n"
"1. **Possible Answers (Previous Query Parts):**\n"
f"- Theory Level: {prev_possible_docs.get('Theory', 'N/A')}\n"
f"- Entity Level: {prev_possible_docs.get('Entity', 'N/A')}\n"
f"- Example Level: {prev_possible_docs.get('Example', 'N/A')}\n\n"
f"- Other Level: {prev_possible_docs.get('Other', 'N/A')}\n\n"

"2. Actual Search Results:\n"
f"{retrieved_docs}\n\n"

"```json\n"
"{\n"
"  \"Reasoning\": \"e.g. Theory 'Poisson' was found in Doc 1. However, Entity 'Submarine' was NOT found; docs discuss 'Bees'.\",\n"
"  \"Actions\": {\n"
"    \"Theory\": \"EXPLOIT\",\n"
"    \"Entity\": \"EXPLORE\",\n"
"    \"Example\": \"EXPLORE\"\n"
"    \"Other\": \"EXPLOIT\"\n"
"  }\n"
"}\n"
"```\n\n"
```

Rewritten query based on predefined levels

Retrieved documents

level마다 어떤 action할지 결정
LLM decide EXPLOIT vs EXPLORE for each level

Agentic search

- Agentic search
  - LLM finds missing abstraction level
- Prompts

  2. Executor

```
if action == 'EXPLOIT':
    instructions_block += (
        f"### {level} Level: ACTION = EXPLOIT (Refine & Deepen)\n"
        f"- **Context:** The hypothesis answer:\n{prev_content}\n was verified in the retrieved docs.\n"
        "- **Instruction:** Refine this content to be more precise. Use key terms from 'Retrieved Documents'.\n"
        …
    )
elif action == 'EXPLORE':
    instructions_block += (
        f"### {level} Level: ACTION = EXPLORE (Pivot & New Hypothesis)\n"
        f"- **Context:** The previous hypothesis \n{prev_content}\n FAILED (not found or irrelevant).\n"
        "- **Instruction:** Identify the common 'wrong direction' or 'missing gap' in the retrieved docs regarding this level. Treat the previous
                    hypothesis as a **Negative Constraint**.\n"
        …
    )
elif action == 'PRUNE':
    instructions_block += f"### {level} Level: ACTION = PRUNE (Remove this level)\n\n"
```

정보량이 많은 **level**에 대해 **refine**

비어있는 **level**에 대해 **rewrite**